Machine Learning II

Lecture 3: Sequences: RNNs, LSTMs and GRUs

Alexander Quispe
ENEI – INEI · PEU-CD 2026

October 12, 2026

Outline

Recap and Motivation

The RNN Idea: Adding Memory

Vanilla RNN Architecture and Math

Backpropagation Through Time

The Vanishing Gradient Problem

LSTM: The Solution

GRU: A Simpler Alternative

Applications in Economics

PyTorch Implementation

Summary and Next Steps

Recap & Motivation

Why do we need RNNs?

Recap

What we learned about CNNs:

- **Inductive biases for images:** locality + translation invariance.
- **Convolution:** learn shared filters that slide across spatial dimensions.
- **Pooling:** downsample and accumulate receptive field.
- **Architectures:** LeNet $\rightarrow$ AlexNet $\rightarrow$ VGG $\rightarrow$ ResNet.
- Worked great for **images** (fixed-size, spatial structure).

But what about *sequential* data?

Text:	"*The Fed raised interest rates today*"
Time series:	GDP, inflation, exchange rates over time
Audio:	speech, music, market signals

All have a **temporal/ordering** structure that CNNs don't naturally capture.

Why CNNs and MLPs Don't Work for Sequences

Three core challenges:

1. **Variable length:** sentences have different numbers of words; time series have different histories. MLPs need a **fixed-size** input.
2. **Order matters:** "*the cat ate the fish*" $\neq$ "*the fish ate the cat*". Bag-of-words representations lose all structure.
3. **Long-range dependencies:** the meaning of a word may depend on something far back in the sentence. Inflation today depends on monetary policy years ago.

The Need for Memory

We need a model with **memory**: a state that summarizes what has been seen so far.

Examples of Sequential Data in Economics

Data type	**Example**	**Task**
Macro time series	GDP, CPI, unemployment	Forecasting, nowcasting
Financial time series	stock prices, FX rates, yields	Volatility, returns
Central bank text	FOMC minutes, ECB statements	Sentiment, policy class.
News articles	Bloomberg, Reuters headlines	Market reaction
Transaction data	credit card streams, payments	Fraud detection
Survey responses	sequential questions	Behavioral modeling

Common thread: order and timing carry information.
RNNs (and later, Transformers) are the modern tools for these tasks.

The RNN Idea: Adding Memory

Core Idea: A Hidden State That Persists

Setup: we receive inputs $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$ one at a time.

Idea: at each step t, maintain a **hidden state** $\mathbf{h}_t$ that summarizes everything seen so far.

The RNN Recurrence

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

where f is a learnable function (a small neural network).

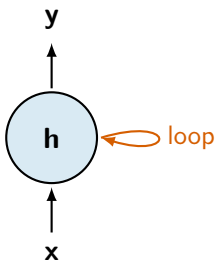
Properties:

- Same function f applied at every time step (**parameter sharing**).
- Handles **variable-length** sequences naturally.
- State $\mathbf{h}_t$ acts as memory of all past inputs $\mathbf{x}_1, \ldots, \mathbf{x}_t$.

Parameter sharing across time is the temporal analogue of weight sharing in CNNs.

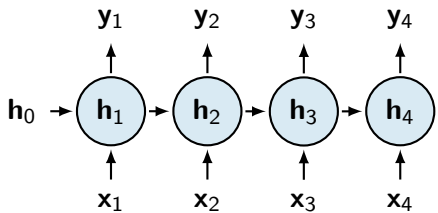
Rolled vs Unrolled View

Rolled (compact):



One block, with a self-loop.

Unrolled (over time):



Same network, replicated across T time steps.

Key insight: the rolled and unrolled views are **equivalent**. The unrolled view makes backpropagation conceptually clear.

All time steps share the same parameters.

Vanilla RNN: Architecture & Math

The Vanilla RNN Equations

Vanilla RNN

Hidden state (the recurrence):

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t + \mathbf{b}_h)$$

Output (the prediction at time t):

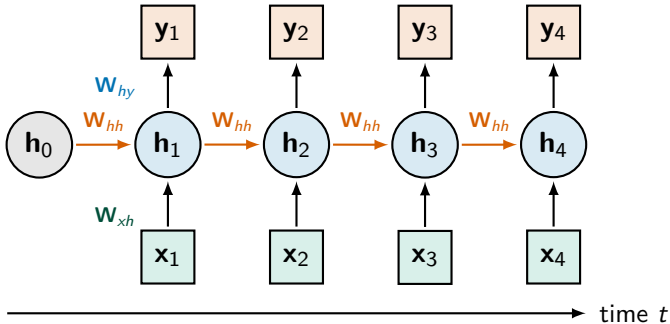
$$\mathbf{y}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y$$

Dimensions: let $d =$ input size, $h =$ hidden size, $k =$ output size.

- $\mathbf{x}_t \in \mathbb{R}^d$, $\mathbf{h}_t \in \mathbb{R}^h$, $\mathbf{y}_t \in \mathbb{R}^k$
- $\mathbf{W}_{xh} \in \mathbb{R}^{h\times d}$, $\mathbf{W}_{hh} \in \mathbb{R}^{h\times h}$ (recurrent), $\mathbf{W}_{hy} \in \mathbb{R}^{k\times h}$

Total parameters: $hd + h^2 + kh + h + k$ — independent of sequence length T.

Unrolled Computational Graph



Each time step uses the *same* weights $(\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy})$. Information flows forward through the red recurrent connections.

Numerical Example: Forward Pass

Tiny RNN: $d = 1,\ h = 2,\ k = 1$. Inputs: $\mathbf{x}_1 = 1, \mathbf{x}_2 = 2, \mathbf{x}_3 = 3$.

Parameters:

$$\mathbf{W}_{xh} = \begin{pmatrix} 0.5 \\ 0.3 \end{pmatrix}, \quad \mathbf{W}_{hh} = \begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{pmatrix}, \quad \mathbf{W}_{hy} = \begin{pmatrix} 0.2 & 0.5 \end{pmatrix}, \quad \mathbf{b}_h = \mathbf{0},\ \mathbf{b}_y = 0$$

Initial state: $\mathbf{h}_0 = \mathbf{0}$.

Time $t = 1$**:**

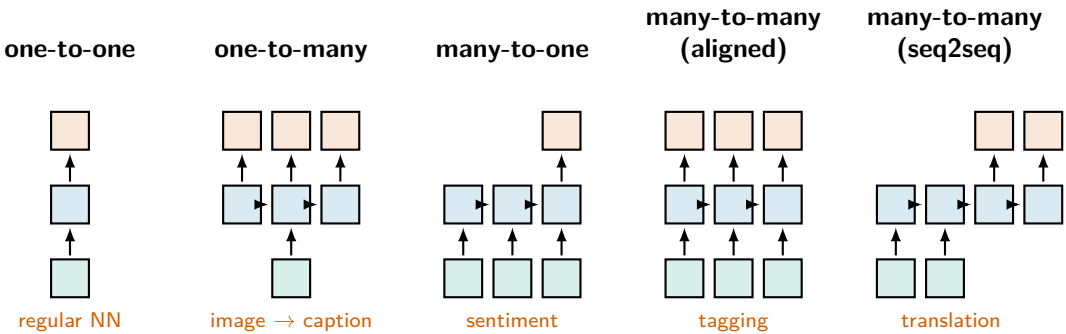
$$\mathbf{h}_1 = \tanh(\mathbf{W}_{hh} \cdot \mathbf{0} + \mathbf{W}_{xh} \cdot 1) = \tanh\begin{pmatrix} 0.5 \\ 0.3 \end{pmatrix} \approx \begin{pmatrix} 0.462 \\ 0.291 \end{pmatrix}$$

Time $t = 2$**:**

$$\mathbf{h}_2 = \tanh\left(\begin{pmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{pmatrix} \begin{pmatrix} 0.462 \\ 0.291 \end{pmatrix} + \begin{pmatrix} 0.5 \\ 0.3 \end{pmatrix} \cdot 2 \right) \approx \begin{pmatrix} 0.847 \\ 0.946 \end{pmatrix}$$

Output at $t = 2$**:** $y_2 = 0.2 \cdot 0.847 + 0.5 \cdot 0.946 = 0.642$.

RNN Architectures (Karpathy's Diagram)



Hidden states (blue), inputs (green), outputs (red).

Backpropagation Through Time (BPTT)

The Loss and Gradient Goal

For a sequence with predictions $\mathbf{y}_1, \ldots, \mathbf{y}_T$ and targets $\hat{\mathbf{y}}_1, \ldots, \hat{\mathbf{y}}_T$:

$$\mathcal{L} = \sum_{t=1}^{T} \mathcal{L}_t(\mathbf{y}_t, \hat{\mathbf{y}}_t)$$

Goal: compute $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{xh}}$, $\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hy}}$ to update the weights.

Challenge: $\mathbf{W}_{hh}$ is used **at every time step**. So:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}}$$

And each $\mathcal{L}_t$ depends on $\mathbf{W}_{hh}$ through **all** hidden states $\mathbf{h}_1, \ldots, \mathbf{h}_t$.

This is just the chain rule applied to the unrolled graph: backpropagation through time.

BPTT: The Chain Rule Through Time

For a single loss $\mathcal{L}_t$ at time t, the gradient w.r.t. $\mathbf{W}_{hh}$ involves **all earlier hidden states**:

BPTT Formula

$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}_{hh}} = \sum_{k=1}^{t} \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \cdot \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \cdot \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$$

where the **Jacobian product** is:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^{t} \frac{\partial \mathbf{h}_i}{\partial \mathbf{h}_{i-1}} = \prod_{i=k+1}^{t} \mathbf{W}_{hh}^{\top} \operatorname{diag}(\tanh'(\mathbf{z}_i))$$

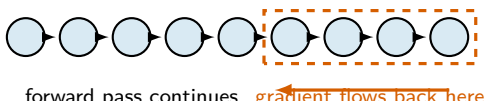
where $\mathbf{z}_i = \mathbf{W}_{hh}\mathbf{h}_{i-1} + \mathbf{W}_{xh}\mathbf{x}_i + \mathbf{b}_h$.

The gradient flowing back from t to k is a **product of $t-k$ Jacobians**.
This product is the source of vanishing/exploding gradients.

Truncated BPTT (Practical Trick)

Problem: for long sequences (e.g., 1000 tokens), backprop through all T steps is expensive and unstable.

Solution: truncated BPTT – only backpropagate through the last K steps (e.g., $K = 35$ or $K = 100$).



Trade-off: can't learn dependencies longer than K steps, but training is feasible.

In PyTorch: use `detach()` on the hidden state every K steps.

The Vanishing/Exploding Gradient Problem

Why Long-Range Dependencies Are Hard

Recall the gradient flowing from time t back to time k:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{i=k+1}^{t} \mathbf{W}_{hh}^{\top} \operatorname{diag}(\tanh'(\mathbf{z}_i))$$

Two facts:

- $\tanh'(z) \in [0, 1]$, with maximum at $z = 0$.
- Multiplying many matrices $\mathbf{W}_{hh}^{\top}$ together $\Rightarrow$ behavior governed by the **spectral radius** (largest eigenvalue magnitude) $\rho(\mathbf{W}_{hh})$.

Spectral Analysis

- If $\rho(\mathbf{W}_{hh}) < 1$: gradient **vanishes** exponentially $\to 0$.
- If $\rho(\mathbf{W}_{hh}) > 1$: gradient **explodes** exponentially $\to \infty$.

Vanishing Gradients: The Product of Jacobians

For $\mathbf{h}_t = \sigma(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b})$, write $\mathbf{a}_t = \mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t + \mathbf{b}$.

One step of the chain rule

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}\big(\sigma'(\mathbf{a}_t)\big)\,\mathbf{W}, \qquad \text{so} \qquad \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} = \prod_{k=t+1}^{T} \operatorname{diag}\big(\sigma'(\mathbf{a}_k)\big)\,\mathbf{W}.$$

Bounding the product

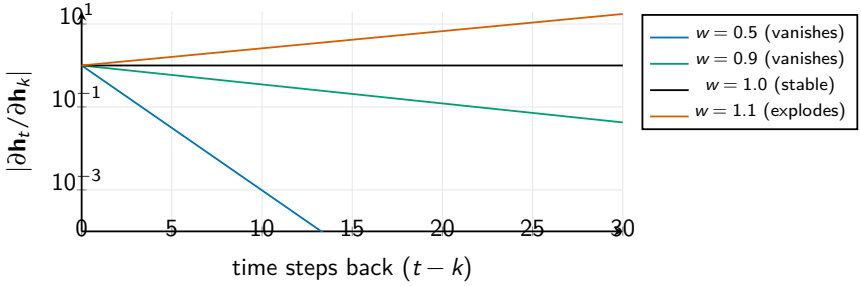
Let $\gamma = \max_z \sigma'(z)$ ($\gamma = 1$ for tanh, $\gamma = \frac{1}{4}$ for the sigmoid) and $\|\mathbf{W}\|_2$ the largest singular value. Sub-multiplicativity of the spectral norm gives

$$\left\| \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_t} \right\|_2 \;\le\; \prod_{k=t+1}^{T} \left\|\operatorname{diag}(\sigma'(\mathbf{a}_k))\right\|_2 \|\mathbf{W}\|_2 \;\le\; \big(\gamma\,\|\mathbf{W}\|_2\big)^{\,T-t}.$$

- If $\gamma\|\mathbf{W}\|_2 < 1$ the bound decays **exponentially** in the time gap $T - t$: the gradient from a loss at time T cannot reach a parameter that mattered at time t. This is the vanishing-gradient problem, and it is unavoidable for a sigmoid ($\gamma = \frac{1}{4}$) unless $\|\mathbf{W}\|_2 > 4$.
- If $\gamma\|\mathbf{W}\|_2 > 1$ nothing prevents **explosion**; gradient clipping is the standard patch.
- The LSTM's cell state has Jacobian $\operatorname{diag}(\mathbf{f}_t)$ — the forget gate — with no $\mathbf{W}$ in the product. That is the whole design.

Visualizing Vanishing Gradients

Setting: $\mathbf{W}_{hh}$ scalar with $w \in \{0.5, 0.9, 1.0, 1.1\}$ (1D RNN, $\tanh' \approx 1$).



Practical consequence: vanilla RNNs cannot learn dependencies more than $\approx$**10–20 time steps** apart.

This is a fundamental limitation, not a training issue.

Partial Fixes (Insufficient)

Gradient clipping (for exploding gradients):

$$\text{if } \|\nabla\| > c, \quad \nabla \leftarrow \frac{c}{\|\nabla\|}\nabla$$

Caps gradient magnitude. Helps with explosion, *not* with vanishing.

Initialization: initialize $\mathbf{W}_{hh}$ as orthogonal or identity matrix. Helps at the start, but breaks down during training.

ReLU instead of tanh: doesn't saturate, but introduces other instabilities.

None of these solve the fundamental problem.

We need a fundamentally different architecture: **LSTM**.

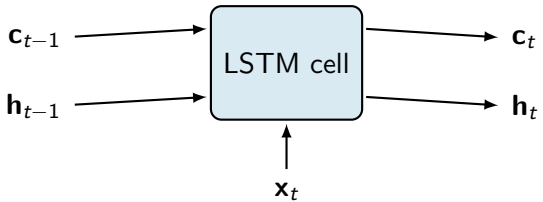
Long Short-Term Memory (LSTM)

Hochreiter & Schmidhuber (1997)

LSTM: The Key Idea

Vanilla RNN problem: information must squeeze through the recurrent path at every step. Repeated multiplications destroy gradients.

LSTM solution: introduce a separate **cell state** $\mathbf{c}_t$ that flows through time *additively*, with **gates** that control information flow.



Two memory streams:

- **Cell state** $\mathbf{c}_t$: the "long-term" memory (highway for gradients).
- **Hidden state** $\mathbf{h}_t$: the "working memory" / output for this time step.

The Three Gates

A gate is a vector in $[0,1]^h$ from a sigmoid. Element-wise multiplication acts as a soft mask: 0 = "forget", 1 = "keep".

Forget gate (what to drop from $\mathbf{c}_{t-1}$):

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

Input gate (what new info to add):

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

Output gate (what part of $\mathbf{c}_t$ to expose as $\mathbf{h}_t$):

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

Candidate value (proposed new info):

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$$

$[\mathbf{h}_{t-1}, \mathbf{x}_t]$ = concatenation, so $\mathbf{W}_f \in \mathbb{R}^{h \times (h+d)}$.

The LSTM Updates

Putting it all together:

LSTM Updates

Cell state update:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

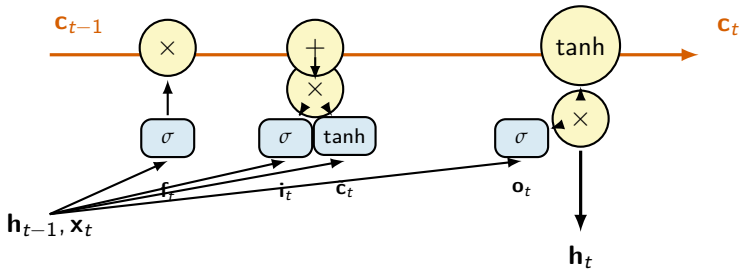
Hidden state:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

Reading the cell state: keep $\mathbf{f}_t \odot \mathbf{c}_{t-1}$ from old memory, add $\mathbf{i}_t \odot \tilde{\mathbf{c}}_t$ new info. The **additive** update is key to gradient flow.

Reading the hidden state: cell state squashed by tanh, then masked by output gate.

LSTM Cell: Visual Diagram



Red highway: cell state path. **Yellow circles:** pointwise operations. **Blue boxes:** learned gates.

Why LSTMs Solve Vanishing Gradients

Gradient flow through the cell state:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathrm{diag}(\mathbf{f}_t)$$

Compare with vanilla RNN:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \mathbf{W}_{hh}^\top \mathrm{diag}(\tanh'(\mathbf{z}_t))$$

The Crucial Insight

Key difference: LSTM gradient is just $\mathbf{f}_t$. If the network learns $\mathbf{f}_t \approx 1$ for relevant steps, gradient flows back **intact** through arbitrarily many time steps.

Practical effect: LSTMs can learn dependencies over **hundreds** of steps, vs ~10-20 for vanilla RNNs.

This insight powered NLP and speech recognition for two decades (1997–2017).

Gated Recurrent Unit (GRU)

Cho et al. (2014)

GRU: Simplified LSTM

Idea: merge cell state and hidden state. Use only **2 gates**.

Reset gate (how much to forget):

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r)$$

Update gate (how much new info):

$$\mathbf{u}_t = \sigma(\mathbf{W}_u[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_u)$$

Candidate:

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h)$$

New hidden state:

$$\mathbf{h}_t = (1 - \mathbf{u}_t) \odot \mathbf{h}_{t-1} + \mathbf{u}_t \odot \tilde{\mathbf{h}}_t$$

LSTM vs GRU

	LSTM	**GRU**
Gates	3 (forget, input, output)	2 (reset, update)
States	2 (cell, hidden)	1 (hidden)
Parameters	more	$\sim$ 25% fewer
Training speed	slower	faster
Performance	slightly better on long seq.	comparable on short/medium
Use cases	language models, long sequences	sentiment, time series

When to use each:

- **LSTM:** when you have plenty of data and need to model long dependencies.
- **GRU:** when you want fewer parameters and faster training (often the better default).

In practice: try both, keep what works. Most time series tasks favor GRU.

Applications in Economics

Time Series Forecasting

Classical: ARIMA, GARCH, VAR. Linear, parametric, easy to interpret.

RNN-based: non-linear, can capture regime shifts and complex dynamics.

Examples in the literature:

- **GDP nowcasting** from heterogeneous data (Bok et al. 2018; Smalter Hall 2018).
- **Inflation forecasting** with LSTMs vs. Phillips curve models.
- **Volatility prediction** – LSTMs vs GARCH for S&P 500 (Kim & Won 2018).
- **Yield curve forecasting** (Nunes et al. 2019).
- **Exchange rate prediction** (limited success – markets are nearly random).

Caveat: RNNs need lots of data. With short macro time series ($T < 200$), classical models are often competitive.

Text Mining for Economics

Workflow: embed words → pass through RNN → classify or regress.

Examples:

- **Central bank sentiment:** Hansen & McMahon (2016) and follow-ups – predict policy stance from FOMC minutes.
- **Earnings call analysis:** predict stock returns from CEO speeches.
- **News sentiment:** forecast market moves from headlines.
- **Regulatory text:** classify SEC filings, EU directives.

Pre-2018 status: LSTMs were the dominant tool for sequence text in economics.
Post-2018: BERT, RoBERTa, and other Transformer-based models replaced RNNs (we'll see in Lectures 6-8).

Many published economics papers *still* use LSTMs — understanding them is essential.

PyTorch Implementation

Building an RNN in PyTorch

```python
import torch
import torch.nn as nn

class SimpleRNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.fc  = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        # x: (batch, seq_len, input_size)
        out, h_n = self.rnn(x)          # out: (batch, seq_len, hidden_size)
        return self.fc(out[:, -1, :])  # use the last hidden state

# Example: predict next-day GDP indicator from 30-day window
model = SimpleRNN(input_size=5, hidden_size=64, output_size=1)
x = torch.randn(32, 30, 5)              # batch=32, seq_len=30, features=5
y_pred = model(x)                        # (32, 1)
```

Just swap `nn.RNN` for `nn.LSTM` or `nn.GRU`!

LSTM for Time Series Prediction

```
class LSTMForecaster(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, num_layers,
                            batch_first=True, dropout=0.2)
        self.fc = nn.Linear(hidden_size, 1)

    def forward(self, x):
        out, (h_n, c_n) = self.lstm(x)
        return self.fc(out[:, -1, :])

# Train on a sliding window of an economic indicator
model = LSTMForecaster()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(100):
    for x_batch, y_batch in loader:
        y_pred = model(x_batch)
        loss = nn.MSELoss()(y_pred, y_batch)
        optimizer.zero_grad(); loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)  # clip!
        optimizer.step()
```

Practical Tips

Always do:

- **Normalize** inputs (mean 0, std 1).
- **Gradient clipping** to handle exploding gradients (`clip_grad_norm_`).
- Use **LSTM** or **GRU**, not vanilla RNN, unless you have a good reason.
- **Truncated BPTT** for long sequences.
- **Bidirectional** RNNs when the entire sequence is available at inference (`bidirectional=True`).

Common pitfalls:

- Forgetting to use the *last* hidden state (or all of them, depending on task).
- Wrong tensor shape: PyTorch defaults to (seq, batch, feature). Use `batch_first=True`.
- Not clipping gradients $\rightarrow$ NaN losses.
- Too small hidden size $\rightarrow$ underfitting; too large $\rightarrow$ overfitting.

Summary & Next Steps

Summary

1. **Sequential data** requires architectures with memory: variable length, ordering, long dependencies.
2. **Vanilla RNN**: $\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{W}_{xh}\mathbf{x}_t)$. Parameter sharing across time.
3. **BPTT**: chain-rule gradients require multiplying t Jacobians.
4. **Vanishing/exploding gradients**: spectral radius of $\mathbf{W}_{hh}$ controls behavior. Limits vanilla RNNs to ~ 10 time steps.
5. **LSTM**: introduces a separate cell state $\mathbf{c}_t$ with additive update $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$. Solves vanishing gradients.
6. **GRU**: simpler, 2 gates, comparable performance.
7. **Applications:** time series forecasting, central bank text analysis, sentiment.

Next Lecture: Attention Mechanisms

Lecture 6 (Saturday, May 2):

- Limitation of RNNs: information bottleneck through a single hidden state.
- Attention mechanism: queries, keys, values.
- Self-attention.
- Multi-head attention.
- Lays the foundation for Transformers (Lecture 7).

The big picture:

$$\textit{Vanilla RNN} \rightarrow \textit{LSTM} \rightarrow \textit{Attention} \rightarrow \textit{Transformer} \rightarrow \textit{LLMs}$$

Recommended reading:

- Karpathy, A. (2015). *The Unreasonable Effectiveness of Recurrent Neural Networks*.
- Hochreiter & Schmidhuber (1997), *Long Short-Term Memory*.
- Olah, C. *Understanding LSTM Networks* (blog).

Questions?

References

- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8).
- Cho, K., et al. (2014). Learning phrase representations using RNN encoder-decoder. *EMNLP*.
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. *ICML*.
- Karpathy, A. (2015). The unreasonable effectiveness of recurrent neural networks. *Blog*.
- Hansen, S., & McMahon, M. (2016). Shocking language: Understanding the macroeconomic effects of central bank communication. *Journal of International Economics*.
- Kim, H. Y., & Won, C. H. (2018). Forecasting the volatility of stock price index. *Expert Systems with Applications*.
- Bok, B., Caratelli, D., Giannone, D., Sbordone, A., & Tambalotti, A. (2018). Macroeconomic nowcasting and forecasting with big data. *Annual Review of Economics*.
- Olah, C. (2015). Understanding LSTM networks. *colah.github.io*.